

CSE 144 Final Project Report

Transfer Learning Challenge

Group members:

Tien Le (Cruz ID: 1962505)

Spring 2026

1. Introduction

For this project, I was tasked with applying transfer learning techniques in order to fine-tune a pretrained model to classify a custom dataset provided by the instructors.

With the limited time span, computing resources, and size of the provided dataset, transfer learning is a particularly appropriate approach, allowing one to take advantage of the work already done on existing models.

To this end, I utilized strategic layer-freezing, various data augmentation techniques, and some architecture changes in order to achieve a __% accuracy on Kaggle.

2. Dataset

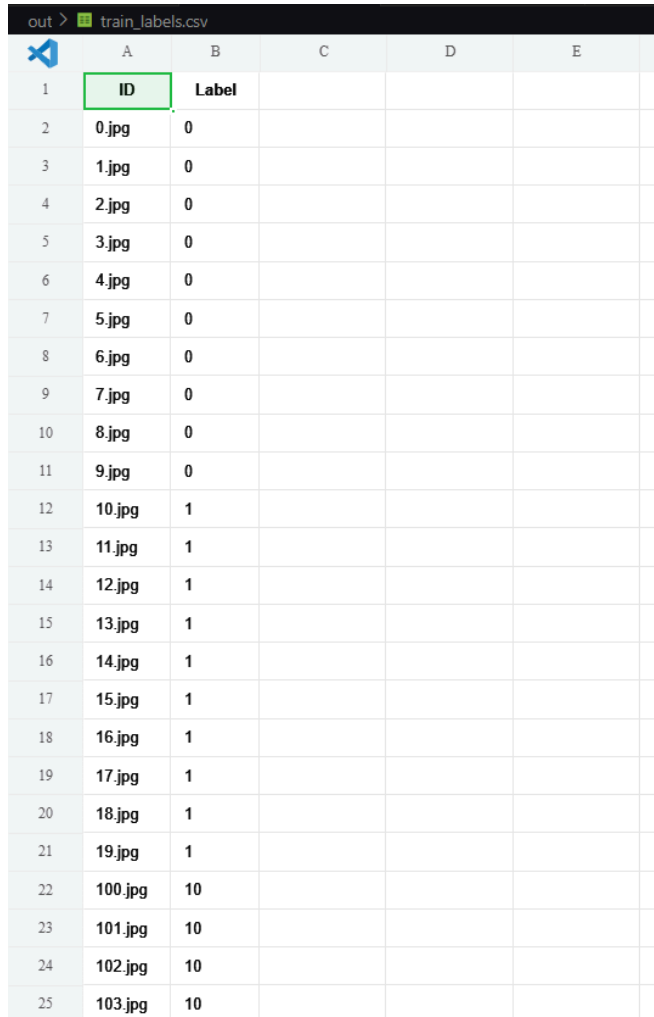
The dataset has the following features:

- Meant to represent 100 classes, spanning from specific classifications of flowers to vehicles to food
- Colored (RGB) images, of varying sizes/resolutions
- 1078 labelled training samples, meaning approximately 10 samples per label, with the number of samples varying from label to label
- 1035 unlabelled test samples

From this, I further split the labelled samples into training and validation datasets at 971 and 108 samples respectively, a roughly 9:1 split.

As provided by the instructors, the labelled training samples are provided in a directory that groups samples into folders by label, i.e. image “40.jpg” is held in folder “train/4”, meaning image 40.jpg’s intended label is 4.

To avoid scrambling labels, I wrote a .csv to act as an annotations file for the dataset, walking through the file directory to match each image file to the folder's name regardless of walk order, thereby ensuring each image has the correct label.



	A	B	C	D	E
1	ID	Label			
2	0.jpg	0			
3	1.jpg	0			
4	2.jpg	0			
5	3.jpg	0			
6	4.jpg	0			
7	5.jpg	0			
8	6.jpg	0			
9	7.jpg	0			
10	8.jpg	0			
11	9.jpg	0			
12	10.jpg	1			
13	11.jpg	1			
14	12.jpg	1			
15	13.jpg	1			
16	14.jpg	1			
17	15.jpg	1			
18	16.jpg	1			
19	17.jpg	1			
20	18.jpg	1			
21	19.jpg	1			
22	100.jpg	10			
23	101.jpg	10			
24	102.jpg	10			
25	103.jpg	10			

Above, a few rows of “train_labels.csv” showing the correctly matched labels.

3. Implementation

3.1 Model

For my pretrained backbone, I used EfficientNetV2-S. As the machine I am training on lacks a GPU and is not particularly strong, EfficientNetV2-S's smaller size at 21.5M parameters would have ideally helped make the training process feasible. This, combined with the relatively strong accuracy it provided due to making use of a modified version of [Torchvision's new training recipe](#) (2021), made it to be what I felt was a prime candidate.

I made a few architecture changes to this model - I swapped out the classifier head.
Beyond the basic task of